

RPAL Interpreter

Lexical Analyzer, Parser, Standardizer & CSE Machine
CS 3513 — Programming Languages — Programming Project

Group Member	Name	Student ID
Student 1		
Student 2		

Language: C++ (C++17) | Build: **make** | Executable: **rpal20**

1. Introduction

This project implements a complete interpreter for the RPAL (Right-reference Pedagogic Algorithmic Language) programming language. The interpreter reads an RPAL source program from a file and processes it through four stages: lexical analysis, parsing into an Abstract Syntax Tree (AST), standardization of the AST into a Standardized Tree (ST), and evaluation on a Control–Stack–Environment (CSE) machine. The output of the interpreter matches that of the reference implementation *rpal.exe*.

The implementation is written entirely in C++ and uses no parser-generator tools such as *lex* or *yacc*; the lexer and parser are written by hand. The lexical rules follow *RPAL_Lex.pdf* and the grammar follows *RPAL_Grammar.pdf*.

2. How to Build and Run

A **Makefile** is placed directly under the project root. The following commands build and run the interpreter:

```
$ make                # builds ./rpal20
$ ./rpal20 file_name  # evaluate an RPAL program
$ ./rpal20 -ast file_name # print the Abstract Syntax Tree
$ ./rpal20 -st file_name # print the Standardized Tree
$ make clean          # remove build artifacts
```

The grader's workflow is fully supported:

```
$ make
$ ./rpal20 rpal_test_programs/rpal_01 > output.01
$ diff output.01 rpal_test_programs/output01.test
```

3. Program Structure

The interpreter is organized into clearly separated modules, each responsible for one stage of the pipeline. The source files are:

Module / File	Responsibility
rpal20.cpp	Entry point; reads input file, drives the pipeline, handles CLI switches.
src/token.h	Token type enumeration and Token structure.
src/lexer.{h,cpp}	Lexical analyzer: source text -> token stream (RPAL_Lex.pdf).
src/node.{h,cpp}	Tree node (first-child / next-sibling) and pre-order printer.
src/parser.{h,cpp}	Recursive-descent parser: tokens -> AST (RPAL_Grammar.pdf).
src/standardizer.{h,cpp}	AST -> Standardized Tree rewrite rules.
src/cse_machine.{h,cpp}	Control-structure generation and CSE machine evaluation.

The four stages communicate through a single tree data structure. The tree uses the classic **first-child / next-sibling** (left-child right-sibling) representation, which is convenient both for printing trees in the RPAL “dotted” pre-order format and for applying the standardization rewrite rules in place.

4. Lexical Analyzer

The lexer scans the input character by character and produces a list of tokens. It recognizes identifiers, integers, strings, operator-symbol sequences, and the punctuation tokens () ; , . Whitespace and // comments are recognized and discarded (the <DELETE> rule). Identifiers that match a reserved word (*let, in, fn, where, aug, or, not, gr, ge, ls, le, eq, ne, true, false, nil, dummy, within, and, rec*) are reclassified as keyword tokens.

Key function prototypes (src/lexer.h):

```
class Lexer {
public:
    explicit Lexer(const std::string &source);
    std::vector<Token> tokenize();
private:
    char peek(int offset = 0) const;
    char advance();
    bool atEnd() const;
    bool isLetter(char c) const;
    bool isDigit(char c) const;
    bool isOperatorSymbol(char c) const;
    Token scanIdentifier();
    Token scanInteger();
    Token scanOperator();
    Token scanString();
    void skipSpaces();
    void skipComment();
};
```

5. Parser and Abstract Syntax Tree

The parser is a hand-written recursive-descent parser. Each non-terminal of the RPAL grammar is implemented as a procedure. Left-recursive productions (for example E, A, At, R, B, Bt, Ta) are rewritten as iterative loops. The AST is built with a node stack: each procedure pushes leaf nodes for terminals and, after recognizing a production with a build action (=> 'label'), pops the appropriate number of children and pushes a new parent node labelled accordingly (the buildTree operation).

Key function prototypes (src/parser.h):

```
class Parser {
public:
    explicit Parser(const std::vector<Token> &tokens);
    Node *parse();
private:
    const Token &peek() const;
    void advance();
    void buildTree(const std::string &label, int n);
    // one procedure per grammar non-terminal:
    void E(); void Ew(); void T(); void Ta(); void Tc();
    void B(); void Bt(); void Bs(); void Bp();
    void A(); void At(); void Af(); void Ap();
    void R(); void Rn();
    void D(); void Da(); void Dr(); void Db();
    void Vb(); void Vl();
};
```

Leaf nodes are labelled in a form that records their lexical category, e.g. <ID:x>, <INT:3>, <STR:'hi'>, <true>, <nil>.

6. Standardizer (AST → Standardized Tree)

The standardizer performs a post-order traversal (children first) and rewrites each non-standard node into its standardized form. The rules applied are:

Node	Standardized form
let(=(X,E), P)	gamma(lambda(X, P), E)
where(P, =(X,E))	gamma(lambda(X, P), E)
fcn_form(F, V1..Vn, E)	=(F, lambda(V1, lambda(V2, ... lambda(Vn, E))))
lambda(V1..Vn, E)	lambda(V1, lambda(V2, ... lambda(Vn, E)))
within(=(X1,E1), =(X2,E2))	=(X2, gamma(lambda(X1, E2), E1))
@(E1, N, E2)	gamma(gamma(N, E1), E2)
and(=(X1,E1)..=(Xn,En))	=(, (X1..Xn), tau(E1..En))
rec(=(X,E))	=(X, gamma(Y*, lambda(X, E)))

Function prototypes (src/standardizer.h):

```
class Standardizer {
public:
    Node *standardize(Node *root);
private:
    void standardizeNode(Node *node);
    int childCount(Node *node) const;
    Node *nthChild(Node *node, int i) const;
};
```

7. CSE Machine

The standardized tree is flattened into a set of **control structures** (delta_0, delta_1, ...) by a pre-order traversal. Each lambda and each conditional (->) spawns new control structures. The machine then runs a loop over a Control list and a Stack, maintaining an Environment tree. It implements the standard CSE evaluation rules, including stacking of literals and closures, function application (creating new environments), tuple construction (tau), conditional selection (beta), built-in operator application, and the recursion rule using Y* and an eta fixed-point object.

Supported built-in functions: Print, Order, Stem, Stern, Conc, Isinteger, Istruthvalue, Isstring, Istuple, Isfunction, Isdummy, Null, ItoS. Supported operators: arithmetic + - * / ** and unary neg; comparison gr ge ls le eq ne (and symbolic > >= < <=); boolean or & not; and tuple aug.

Function prototypes (src/cse_machine.h):

```
class CSEMachine {
public:
    explicit CSEMachine(Node *standardizedRoot);
    void run();
    bool producedOutput() const;
private:
    void buildControlStructures();
    void preOrder(Node *node, int currentDelta);
    CSElement makeName(Node *node);
    void evaluate();
    bool applyBuiltin(const std::string &fn, std::vector<CSElement> &stack);
};
```

```
CSElement applyUnary(const std::string &op, const CSElement &rand);
CSElement applyBinary(const std::string &op,
                      const CSElement &a, const CSElement &b);
std::string valueToString(const CSElement &v);
};
```

8. Testing and Results

The interpreter was tested on the example given in the project description and on a battery of additional programs covering arithmetic, conditionals, single- and multi-parameter functions, anonymous functions, tuples and tuple selection, string built-ins, `within` and `and` definitions, and recursion (including recursion over tuple-destructured parameters). Selected results:

Program	Output
<code>let Sum(A)=Psum(A,Order A) where rec Psum(T,N)=... (project example)</code>	15
<code>let rec fact n = n eq 0 -> 1 n*fact(n-1) in Print(fact 10)</code>	3628800
<code>let rec fib n = n le 1 -> n fib(n-1)+fib(n-2) in Print(fib 10)</code>	55
<code>let rec gcd a b = b eq 0 -> a gcd b (a-(a/b)*b) in Print(gcd 48 36)</code>	12
<code>let abs n = n ls 0 -> -n n in Print(abs(-7), abs 7)</code>	(7, 7)
<code>Print(Conc 'ab' 'cd')</code>	abcd
<code>let rec Reverse S = S eq '' -> '' Conc(Reverse(Stern S))(Stem S)</code>	olleh
<code>Print((1, (2,3), 'hi', true))</code>	(1, (2, 3), hi, true)

All five bundled test programs in `rpal_test_programs/` produce output that matches their `.test` expected-output files under `diff`.

9. Conclusion

The project delivers a working RPAL interpreter that correctly performs lexical analysis, parsing, standardization, and CSE-machine evaluation. The modular design keeps each stage independent and testable, and the optional `-ast` and `-st` switches make it easy to inspect the intermediate trees. The output conforms to the format of the reference *rpal.exe*.